

Requisições HTTP com Express.js

Entendendo como o servidor recebe e responde requisições

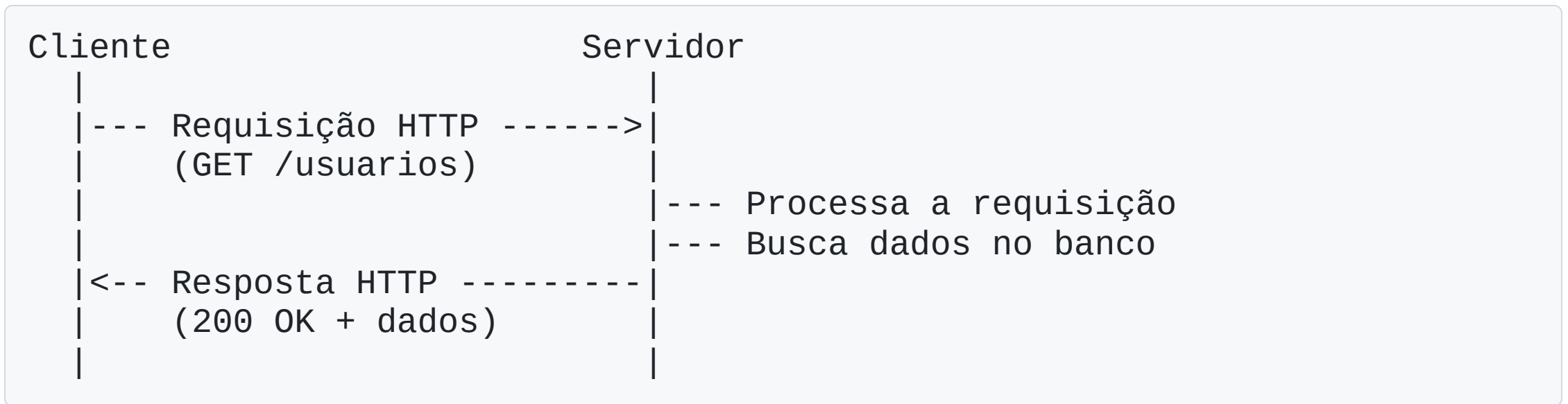
O que são Requisições HTTP?

Uma **requisição HTTP** é a forma como o **cliente** (navegador, aplicativo, Postman) se comunica com o **servidor**. Ela contém:

- **Método** (GET, POST, PUT, DELETE)
- **URL** (o endereço do recurso)
- **Headers** (metadados da requisição)
- **Body** (dados enviados, quando necessário)

O servidor **recebe** a requisição, **processa** e devolve uma **resposta**.

Fluxo de uma Requisição



O Express.js é o framework que **facilita** esse processo no lado do servidor com Node.js.

O que é o Express.js?

O **Express.js** é o framework web mais popular para **Node.js**. Ele simplifica a criação de servidores e APIs.

Por que usar o Express?

- **Minimalista:** Não impõe regras rígidas
- **Flexível:** Permite criar APIs e aplicações web
- **Middleware:** Sistema de plugins para estender funcionalidades
- **Rotas:** Organização clara dos endpoints
- **Comunidade:** Enorme ecossistema de pacotes

Instalando o Express

```
# Inicializa o projeto
npm init -y

# Instala o Express
npm install express
```

Primeiro Servidor

```
const express = require('express');
const app = express();

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```

Criando a Primeira Rota

```
const express = require('express');
const app = express();

// Rota GET para a página inicial
app.get('/', (req, res) => {
  res.send('Bem-vindo ao servidor Express!');
});

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```

Acessando `http://localhost:3000` , o navegador exibe a mensagem.

A Função Callback: req e res

Toda rota no Express recebe uma **função callback** com dois parâmetros:

```
app.get('/rota', (req, res) => {  
  // req = Request (requisição do cliente)  
  // res = Response (resposta do servidor)  
});
```

O objeto `req` (Request)

O `req` contém todas as informações **enviadas pelo cliente**:

Propriedade	Descrição
<code>req.params</code>	Parâmetros da URL (rota dinâmica)
<code>req.query</code>	Parâmetros de consulta (?chave=valor)
<code>req.body</code>	Corpo da requisição (POST, PUT)
<code>req.headers</code>	Cabeçalhos da requisição
<code>req.method</code>	Método HTTP utilizado
<code>req.url</code>	URL completa da requisição
<code>req.ip</code>	IP do cliente

O objeto `res` (Response)

O `res` é usado para **enviar a resposta** ao cliente:

Método	Descrição
<code>res.send()</code>	Envia texto, HTML ou Buffer
<code>res.json()</code>	Envia um objeto JSON
<code>res.status()</code>	Define o código de status HTTP
<code>res.redirect()</code>	Redireciona para outra URL
<code>res.sendFile()</code>	Envia um arquivo
<code>res.render()</code>	Renderiza uma view/template

Exemplo Prático: req e res

```
const express = require('express');
const app = express();

app.get('/saudacao', (req, res) => {
  // Lendo dados da requisição
  console.log('Método:', req.method);           // GET
  console.log('URL:', req.url);                 // /saudacao
  console.log('IP:', req.ip);                   // ::1

  // Enviando resposta
  res.status(200).json({
    mensagem: 'Olá, mundo!',
    status: 'sucesso'
  });
});

app.listen(3000);
```

Query Params (?chave=valor)

Query Parameters são parâmetros passados na URL após o símbolo `?`.

Formato:

```
http://localhost:3000/busca?nome=João&idade=25
```

Acessando no Express:

```
app.get('/busca', (req, res) => {  
  const nome = req.query.nome;    // "João"  
  const idade = req.query.idade;  // "25"  
  
  res.json({  
    nome: nome,  
    idade: idade  
  });  
});
```

Quando usar Query Params?

Query params são ideais para:

- **Filtros:** `/produtos?categoria=eletronicos`
- **Ordenação:** `/produtos?ordenar=preco`
- **Paginação:** `/produtos?page=2&limit=10`
- **Busca:** `/buscar?termo=notebook`
- **Parâmetros opcionais:** dados que podem ou não estar presentes

Exemplo Completo:

```
app.get('/produtos', (req, res) => {  
  const categoria = req.query.categoria;  
  const page = req.query.page || 1;  
  const limit = req.query.limit || 10;  
  
  res.json({ categoria, page, limit });  
});
```


Route Params (/recurso/:id)

Route Parameters são parâmetros definidos **diretamente na URL** da rota.

Formato:

```
http://localhost:3000/usuarios/42
```

Definindo no Express:

```
app.get('/usuarios/:id', (req, res) => {  
  const id = req.params.id; // "42"  
  
  res.json({  
    mensagem: `Buscando usuário com ID: ${id}`  
  });  
});
```

Múltiplos Route Params

Você pode usar **vários** parâmetros em uma mesma rota:

```
// Rota com dois parâmetros
app.get('/usuarios/:userId/posts/:postId', (req, res) => {
  const userId = req.params.userId;
  const postId = req.params.postId;

  res.json({
    usuario: userId,
    post: postId
  });
});

// Acessando: /usuarios/5/posts/100
// Resposta: { "usuario": "5", "post": "100" }
```

Query Params vs Route Params

Característica	Query Params	Route Params
Sintaxe	?chave=valor	/:parametro
Posição	Após ? na URL	Na própria rota
Obrigatório?	Não (opcional)	Sim (faz parte da rota)
Uso ideal	Filtros, busca, paginação	Identificar recursos específicos
Exemplo	/produtos?page=2	/produtos/42

Métodos HTTP no Express

O Express oferece um método para cada verbo HTTP:

```
// GET - Buscar dados
app.get('/usuarios', (req, res) => { ... });

// POST - Criar novo recurso
app.post('/usuarios', (req, res) => { ... });

// PUT - Atualizar recurso completo
app.put('/usuarios/:id', (req, res) => { ... });

// PATCH - Atualizar parcialmente
app.patch('/usuarios/:id', (req, res) => { ... });

// DELETE - Remover recurso
app.delete('/usuarios/:id', (req, res) => { ... });
```

Exemplo: CRUD Completo com Express

```
const express = require('express');
const app = express();
app.use(express.json()); // Para interpretar JSON no body

let usuarios = [
  { id: 1, nome: 'Ana' },
  { id: 2, nome: 'Carlos' }
];

// LISTAR todos
app.get('/usuarios', (req, res) => {
  res.json(usuarios);
});

// BUSCAR por ID
app.get('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const usuario = usuarios.find(u => u.id === id);

  if (!usuario) {
    return res.status(404).json({ erro: 'Usuário não encontrado' });
  }
  res.json(usuario);
});
```

Exemplo: CRUD Completo (continuação)

```
// CRIAR
app.post('/usuarios', (req, res) => {
  const novoUsuario = {
    id: usuarios.length + 1,
    nome: req.body.nome
  };
  usuarios.push(novoUsuario);
  res.status(201).json(novoUsuario);
});

// ATUALIZAR
app.put('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const usuario = usuarios.find(u => u.id === id);

  if (!usuario) {
    return res.status(404).json({ erro: 'Usuário não encontrado' });
  }
  usuario.nome = req.body.nome;
  res.json(usuario);
});

// DELETAR
app.delete('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  usuarios = usuarios.filter(u => u.id !== id);
  res.status(204).send();
});
```

Entendendo o `req.body`

Para enviar dados no **corpo** da requisição (POST, PUT, PATCH), precisamos de um **middleware**:

```
// Middleware para interpretar JSON
app.use(express.json());

// Middleware para interpretar formulários
app.use(express.urlencoded({ extended: true }));
```


Exemplo com POST:

```
app.post('/cadastro', (req, res) => {  
  const { nome, email, senha } = req.body;  
  
  console.log('Nome:', nome);  
  console.log('Email:', email);  
  
  res.status(201).json({  
    mensagem: 'Cadastro realizado com sucesso!',  
    dados: { nome, email }  
  });  
});
```

O que é um Middleware?

Middleware é uma função que executa **no meio** do caminho entre a requisição e a resposta.

```
// Middleware global - executa em TODAS as rotas
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next(); // Passa para o próximo middleware/rota
});

// Middleware específico - executa apenas em uma rota
app.get('/admin', verificarAutenticacao, (req, res) => {
  res.send('Painel administrativo');
});

function verificarAutenticacao(req, res, next) {
  if (req.headers.authorization) {
    next(); // Usuário autenticado, continua
  } else {
    res.status(401).json({ erro: 'Não autorizado' });
  }
}
```

Organizando Rotas com Router

Para projetos maiores, podemos separar as rotas em arquivos:

routes/usuarios.js

```
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.json({ mensagem: 'Lista de usuários' });
});

router.get('/:id', (req, res) => {
  res.json({ mensagem: `Usuário ${req.params.id}` });
});

module.exports = router;
```

Organizando Rotas com Router (continuação)

app.js

```
const express = require('express');
const app = express();
const usuariosRouter = require('./routes/usuarios');

app.use(express.json());

// Prefixo /usuarios para todas as rotas do arquivo
app.use('/usuarios', usuariosRouter);

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```

Agora as rotas ficam organizadas:

- GET /usuarios → Lista todos

Resumo: Ciclo de Vida de uma Requisição

1. Cliente envia requisição
- ↓
2. Express recebe a requisição
- ↓
3. Middlewares globais executam (`app.use`)
- ↓
4. Rota correspondente é encontrada
- ↓
5. Middlewares da rota executam
- ↓
6. Callback da rota processa (`req, res`)
- ↓
7. Resposta é enviada ao cliente (`res.json`, `res.send`, etc.)

Resumo dos Conceitos

Conceito	Descrição
Express	Framework web para Node.js
req (Request)	Objeto com dados da requisição do cliente
res (Response)	Objeto para enviar resposta ao cliente
Query Params	Parâmetros opcionais: <code>?chave=valor</code>
Route Params	Parâmetros obrigatórios: <code>/:parametro</code>
Middleware	Funções intermediárias no ciclo req/res
Router	Módulo para organizar rotas em arquivos separados
req.body	Dados enviados no corpo da requisição

Referências

- [Documentação oficial do Express](#)
- [Guia de roteamento do Express](#)
- [Using Express Router](#)
- [Node.js Documentation](#)